HCLTECH CAREER SHAPER PROGRAMME

III YEAR INDUSTRY PROJECT | ACADEMIC YEAR 2025-26

AI-BASED RESUME SCREENING AND CANDIDATE SHORTLISTING SYSTEM

PROJECT REPORT | PROJECT NO. 34

Domain: Digital-Data-AI / AIML | Difficulty: Advanced | Duration: 4 Weeks

SUBMISSION FIELD	DETAILS
Learner Name	Tanishq Tyagi
Roll Number	2300320100262
College / Campus	ABES Engineering College, Ghaziabad (4th Year)
Mentor Name	Kathiresan
Submission Date	30 October 2026
Project Format	Individual project

GITHUB REPOSITORY

github.com/Tan-hue-123/ai-resume-shortlist

Executive Summary

This project implements a local Flask prototype for resume-to-job-description matching with an explicit human-review step. A recruiter uploads a text-based PDF or UTF-8 TXT resume, selects a job family, and supplies a job description. The application extracts text, detects role-relevant skill phrases, calculates TF-IDF cosine similarity, combines the two signals into an advisory fit score, and stores a compact review record in SQLite. Reviewers can record a decision and rationale; the application also exposes CSV export and JSON endpoints.

The repository contains a 5,000-row synthetic feature dataset, data dictionary, four EDA charts, a separate logistic-regression baseline, and five passing unit tests. The current live score is a hand-set 65/35 heuristic, not the logistic-regression model and not a calibrated hiring probability. All dataset metrics are synthetic and must not be interpreted as evidence of real candidate quality or hiring performance.

IMPLEMENTATION / EVIDENCE NOTE
The report describes the supplied GitHub ZIP as it exists. The Flask application was installed from requirements.txt and run locally (Python development server) on 30 October 2026. The UI figures in Section 8 are real browser captures of that running application, taken with fictional resumes only; the live-run checks are listed in Section 8.1. The two review records shown come from those test uploads.

1. Project Overview

1.1 Problem statement

Recruitment teams must compare many resumes with role-specific requirements, often across inconsistent document formats and writing styles. Manual review can be slow, while simple keyword filters can miss context and transferable skills. The project provides an explainable triage aid that surfaces job-related text evidence and leaves the final decision with a human reviewer.

1.2 Business objective

The objective is to demonstrate a working prototype that accepts resume documents and job descriptions, generates an inspectable matching signal, and supports a documented human review workflow. The project brief also requires measurable validation, EDA with at least three visualizations, dataset documentation, and an ethical review that addresses demographic bias.

1.3 Objectives and scope

- Implement PDF/TXT intake, input validation, text extraction, role-specific skill matching, TF-IDF similarity, and an explainable advisory score.
- Keep the raw resume file and extracted text out of persistent storage; save only a small review record and human decision fields.
- Retain the supplied eight-column dataset schema, extend it to 5,000 synthetic rows, document every column, and run reproducible EDA.
- Evaluate the live scorer with deterministic examples and timing checks, while evaluating the separate classifier only on the synthetic feature dataset.
- Document fairness limitations and a small name/pronoun counterfactual perturbation test without claiming a demographic fairness certification.

1.4 Implemented functionality

MODULE	WHAT THE REPOSITORY IMPLEMENTS
Resume intake	PDF and TXT file upload; allow-list validation; 5 MB maximum; rejects short extracted text.
Text extraction	UTF-8 text decoding; PDF extraction through pdfplumber (which uses pdfminer.six); in-memory processing.
Requirement matching	Case-insensitive literal skill/phrase checks against an editable ten-role skill taxonomy.
Text similarity	scikit-learn TF-IDF with unigram/bigram features and cosine similarity.
Review workflow	Candidate detail page; reviewer decision options; rationale; SQLite persistence; CSV export.
API	GET /api/health and stateless POST /api/score routes are implemented in app.py.
Offline analysis	Pandas EDA script and a separate scikit-learn logistic-regression baseline.

2. System Architecture

The current implementation is a compact, single-process Flask application. The live path and offline modelling path are intentionally separate: the live upload route calls the heuristic scorer in src/screening.py; train_baseline.py is a reproducible experiment on the synthetic CSV and is not called by the upload route.

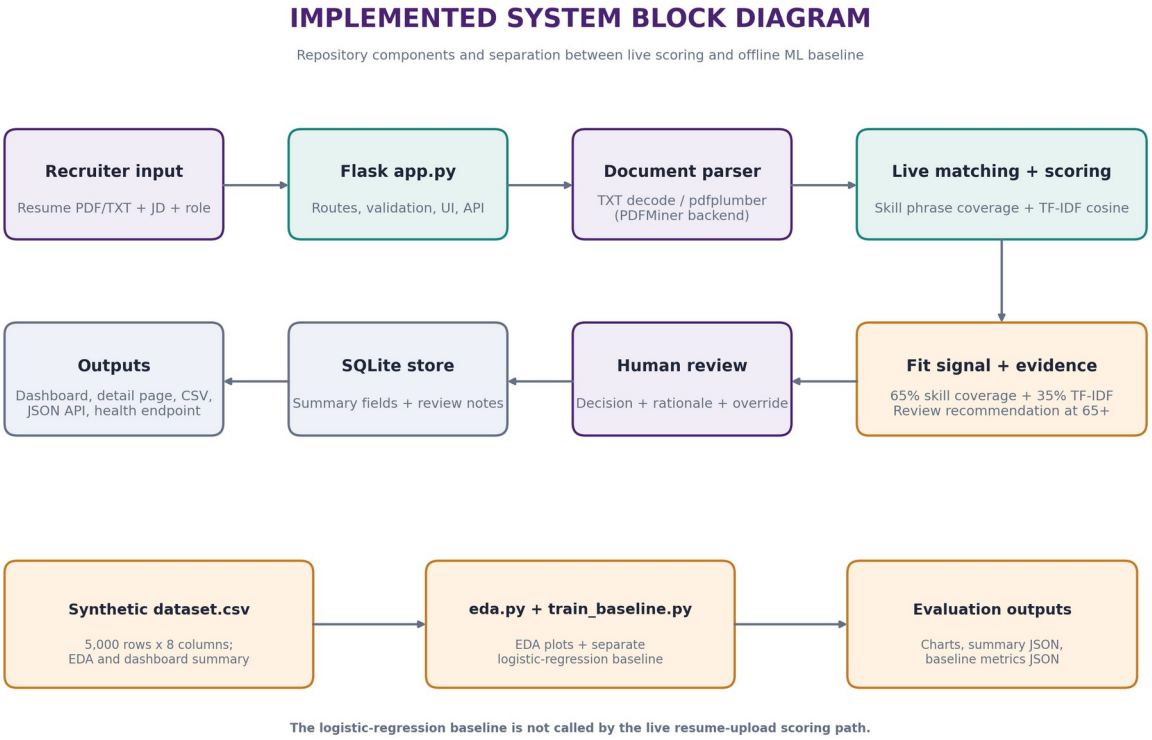


Figure 1. Implemented system block diagram; the bottom lane is offline dataset analysis and is not part of live resume scoring.

2.1 Component responsibilities

COMPONENT	IMPLEMENTED RESPONSIBILITY
app.py (Flask)	Routes for dashboard, upload, candidate review, fairness page, CSV export, health check and stateless scoring API.
src/screening.py	PDF text extraction, experience phrase parsing, taxonomy term matching, TF-IDF similarity and advisory score.
src/db.py	SQLite schema and create/read/list/update helpers for review records.
templates/ + static/	Server-rendered HTML/Jinja pages and CSS for dashboard, screening, candidates, candidate detail and fairness.
eda.py / train_baseline.py	Dataset checks and plots; a separate offline logistic-regression evaluation on synthetic feature vectors.

Technology correction: spaCy is not imported or used by the repository source. The current implementation uses literal phrase matching in Python plus scikit-learn TF-IDF. The brief names spaCy as a suggested tool, but this report does not claim spaCy-based parsing or rule matching.

3. Data Flow Diagram - Level 0

The context diagram shows the system boundary, the recruiter/resume inputs, the SQLite review store, and the separate synthetic dataset input. The final hiring decision is not produced by the scoring module; a reviewer records a decision after examining the evidence.

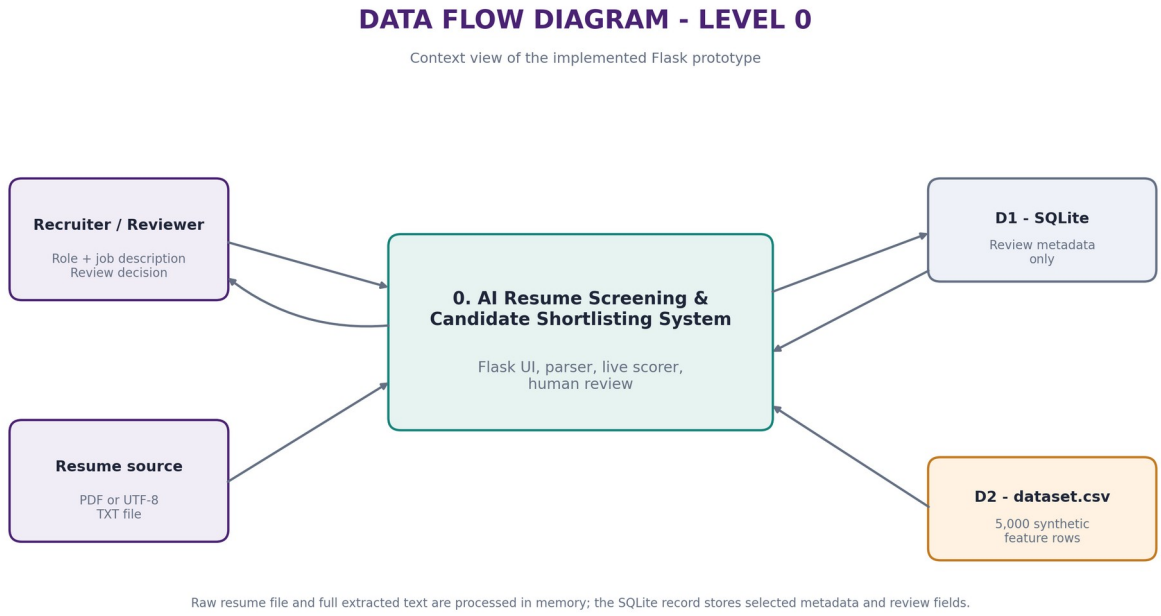


Figure 2. Data Flow Diagram Level 0 for the repository implementation.

- Recruiter to system: target role, job description, and (after review) decision and rationale; system to recruiter: fit signal, matched/missing evidence and exportable review record.
- Resume source to system: PDF or UTF-8 TXT file. The uploaded file and extracted text remain in memory.
- System to/from D1 SQLite: compact candidate summary and review fields are written; saved records are read for the review page and export.
- D2 dataset.csv to system: synthetic feature rows support dashboard summary, EDA and offline baseline analysis; they are not used by the live resume scoring function.

3.1 End-to-end workflow

- A recruiter supplies a PDF/TXT resume, target job role, and job description through the screening form.
- The application checks file extension and size. TXT files are decoded as UTF-8; PDF text is extracted in memory with pdfplumber. Scanned/image-only PDFs are not OCR-processed.
- The scorer chooses requirements detected in the job description when at least three known skill terms are found; otherwise it uses the selected role vocabulary.
- It returns matched and not-detected skills, skill coverage, TF-IDF cosine similarity, a composite score, and a review recommendation.
- A summary record is saved in SQLite. The original uploaded file and full extracted text are not stored. A human reviewer can record one of three decisions and add notes.
- The dashboard reads the synthetic CSV summary; EDA and the logistic-regression baseline run offline from scripts.

3.2 Data storage and privacy boundary

The SQLite record stores a sanitized filename label, role, years-of-experience estimate (when detected), score components, matched/missing terms, review status, reviewer decision, and notes. Filename labels and reviewer notes may still contain personal information. The repository recommends using fictional examples only and deleting the local database when it is no longer required.

4. Implementation and Technology Stack

4.1 Technology stack used by the repository

LAYER	TECHNOLOGY / CURRENT STATUS
Frontend / UI	Flask-rendered Jinja templates, HTML and CSS. No separate JavaScript frontend framework.
Backend / logic	Python 3.10+, Flask application, route and validation logic.
Document extraction	pdfplumber for text-based PDFs (pdfminer.six underneath); UTF-8 decode for TXT.
NLP / matching	Editable role skill dictionaries, literal phrase matching, scikit-learn TfidfVectorizer and cosine similarity. spaCy is not used in this codebase.
Machine learning	scikit-learn logistic-regression baseline is offline only; not wired into live upload scoring.
Database	SQLite for candidate summary and human review records.
Data / visualisation	Pandas and Matplotlib.
Hardware	No special hardware; ordinary development workstation.

4.2 Live scoring logic

The repository computes the live fit signal as:

COMPONENT	DEFINITION	WEIGHT
Required-skill coverage	Number of selected skill phrases detected in resume text divided by the number of selected phrases.	65 points
TF-IDF cosine similarity	Cosine similarity between TF-IDF vectors for resume text and job description, using English stop words and 1-2 grams.	35 points

Fit signal = 65 x skill coverage + 35 x TF-IDF cosine similarity

The result is clipped to [0, 100]. A score at or above 65 produces the UI label “Review for shortlist”; below 65 produces “Review evidence”. Neither outcome is an automatic hiring decision. The field is named predicted_score in the code/database for compatibility, but it is a heuristic score, not a probability. Experience extraction is informational and does not affect the score.

4.3 Parsing and known technical limits

- Only PDF and TXT uploads are accepted; file size is limited to 5 MB.
- pdfplumber extracts selectable text; scanned PDFs or unusual layouts may fail because OCR is not implemented.
- Literal matching can miss synonyms, paraphrases and contextual evidence; punctuation-heavy terms are handled with substring checks.
- TF-IDF is lexical similarity, not semantic understanding or a measure of candidate competence.
- The repository includes spaCy nowhere in requirements.txt or source imports; therefore no spaCy feature is claimed.

5. Dataset Profile and Methodology

5.1 Dataset profile

The CSV follows the exact eight-column schema in the project brief. It contains the original 500 supplied records plus 4,500 generated synthetic feature vectors; the original seed rows and labels are retained. The final file has 5,000 rows across ten job-role families. No real resume text or demographic fields are included.

VALIDATION ITEM	OBSERVED RESULT
Rows / columns	5,000 / 8
Job-role families	10
Missing cells / duplicate resume IDs	0 / 0
Synthetic shortlisted labels	1,316 (26.32%)
Mean / median simulated score	57.11 / 56.30
Years of experience range	0-15 years
Skills-matched range	3-15
Education score range	60-100
Keyword density range	0.10-0.90

5.2 Column definitions

COLUMN	TYPE / RANGE	MEANING
resume_id	Unique string R0001-R5000	Synthetic record key; not a real-person identifier.
job_role	Category; 10 values	Role family used for grouping and one-hot encoding.
years_exp	Integer; 0-15	Synthetic experience proxy.
skills_matched	Integer; 3-15	Synthetic count of matched skills.
education_score	Integer; 60-100	Synthetic education-fit proxy; job relevance needs review.
keyword_density	Float; 0.1-0.9	Synthetic normalized keyword fraction.
predicted_score	Float; 0-100	Provided/simulated score; excluded from baseline predictors.
shortlisted	Binary 0/1	Synthetic label; not a real hiring outcome.

The added 4,500 rows use bounded feature values and synthetic target generation. The original 500 rows are preserved even though their score-to-label relationship is less consistent. Approximately 3% of added labels are deliberate label-noise cases as documented in the repository report materials. The dataset is suitable for demonstrating data handling and evaluation code, not for workforce inference.

6. Exploratory Data Analysis

EDA was rerun from eda.py against data/dataset.csv. The script validates the requested column order, uniqueness, missingness, numeric bounds and binary target, then writes four Matplotlib figures and eda_summary.json. No validation issues were reported.

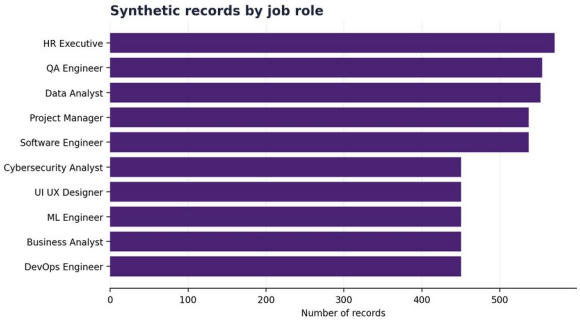


Figure 3. Records by role (n=5,000).

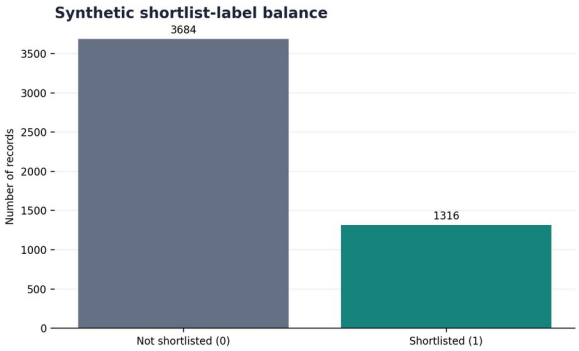


Figure 4. Synthetic label balance.

6.1 EDA observations

All ten role families have records. Counts are not equal because the 500-row seed already contains five role families, while additional roles were added synthetically. Role counts are: Business Analyst: 450; Cybersecurity Analyst: 450; Data Analyst: 552; DevOps Engineer: 450; HR Executive: 570; ML Engineer: 450; Project Manager: 537; QA Engineer: 554; Software Engineer: 537; UI UX Designer: 450. This is a constructed coverage choice, not a representative hiring-market distribution.

The positive label rate is 26.3% (1,316 of 5,000 rows), so the target is moderately imbalanced. Accuracy alone should not be used to describe the baseline; precision, recall, F1 and the confusion matrix are also reported.

6. Exploratory Data Analysis (continued)

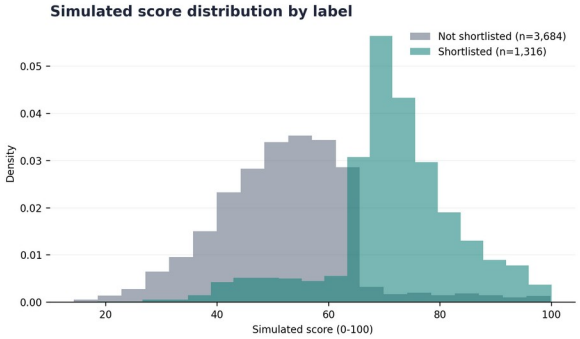


Figure 5. Simulated score distribution by synthetic label.

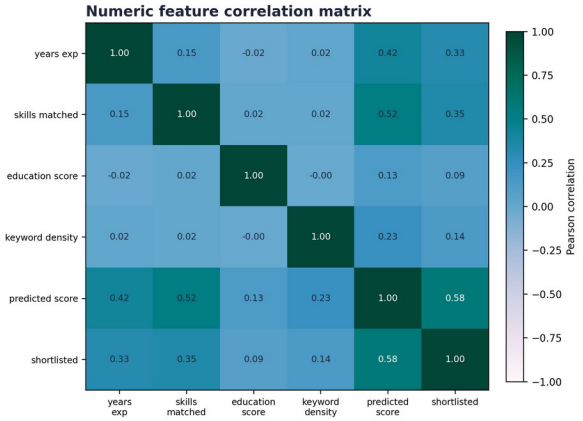


Figure 6. Pearson correlations across numeric features.

6.2 Interpretation and caution

The mean simulated score is 57.11. Records labelled shortlisted have a higher mean score than records labelled not shortlisted; the Pearson correlation between the CSV predicted_score field and shortlisted is 0.585. The two distributions overlap, reflecting the retained seed records and the deliberate synthetic label noise.

The correlation matrix also reflects the synthetic generation recipe: experience, skill-match count and the simulated score have stronger associations with the target than some of the other features. Such patterns are properties of the data-generation process. They do not show that years of experience or an education proxy cause hiring success, and they do not justify using these fields as selection rules.

EDA conclusion: the CSV is structurally complete and adequate for a reproducible coursework demonstration. The distributions should not be generalized to actual applicants, protected groups, job markets, or the performance of the live text scorer.

7. Evaluation and Testing

7.1 Live scorer example and operational checks

The sample files included in examples/ were evaluated by directly calling score_resume() from src/screening.py. The sample resume produced 7 matched skills out of 7 requirements (100% skill coverage), TF-IDF cosine similarity of 0.1988, a skill component of 65.00, a text component of 6.96, and a total fit signal of 72.0/100. The application recommendation was “Review for shortlist”; a reviewer decision was still pending.

LIVE SCORER CHECK	OBSERVED RESULT
Sample resume / JD	7 of 7 selected skills found; no missing terms in this sample.
Fit signal	72.0 / 100; threshold recommendation is advisory only.
Direct function timing	150 in-process calls: median 2.45 ms, p95 3.36 ms.
Timing scope	Scoring function only; excludes Flask request handling, file upload, PDF extraction, database writes and browser latency.
Human decision	Not inferred from score; reviewer action is separately captured.

7.2 Dataset score-label relationship (not live-scorer accuracy)

On the synthetic CSV, the stored/simulated predicted_score field has Pearson r = 0.585 with shortlisted. Treating this dataset field as a simple score threshold at 65 gives accuracy 90.9%, precision 80.5%, recall 86.1%, and F1 83.2%. Confusion matrix: TN=3,410, FP=274, FN=183, TP=1,133.

This is a retrospective association check on the CSV column, not an evaluation of the live upload scorer: dataset predicted_score values are generated/simulated and do not come from running resume text through score_resume(). It is therefore not evidence that the live 65/35 formula achieves these metrics.

7.3 Separate logistic-regression baseline

METRIC	HELD-OUT VALUE	INTERPRETATION
Test examples	1,000	20% stratified holdout; random_state=42.
Accuracy	81.1%	Overall correct predictions.
Precision	72.8%	Positive predictive value on synthetic labels.
Recall	44.9%	Only this share of synthetic positives was found.
F1	55.5%	Precision/recall balance.
ROC-AUC	83.0%	Threshold-independent separation on this split.

The baseline uses job_role, years_exp, skills_matched, education_score and keyword_density; predicted_score is excluded to reduce target leakage. The 81.1% accuracy is from this separate offline classifier and is not produced by the live application.

7.4 Unit-test results

The repository test suite was rerun with python -m unittest discover -s tests -v. All 5 tests passed: explainable score fields, short-text rejection, safe unknown-role fallback, dataset schema/quality, and saving/updating a human review in a temporary SQLite database. These unit tests do not constitute a browser-level or live Flask endpoint integration test.

The manual live run of the web application (page rendering, TXT and PDF upload, human review, CSV export and JSON API) is recorded separately in Section 8.1.

8. Project UI Screenshots - Running Application

The figures in this section are real browser captures of the Flask application running locally (python app.py, 30 October 2026), rendered by the application itself. Only fictional test material was used: examples/sample_resume.txt and a fictional text-based PDF resume. The two review records visible in the figures (#1 and #2) were created by those test uploads.

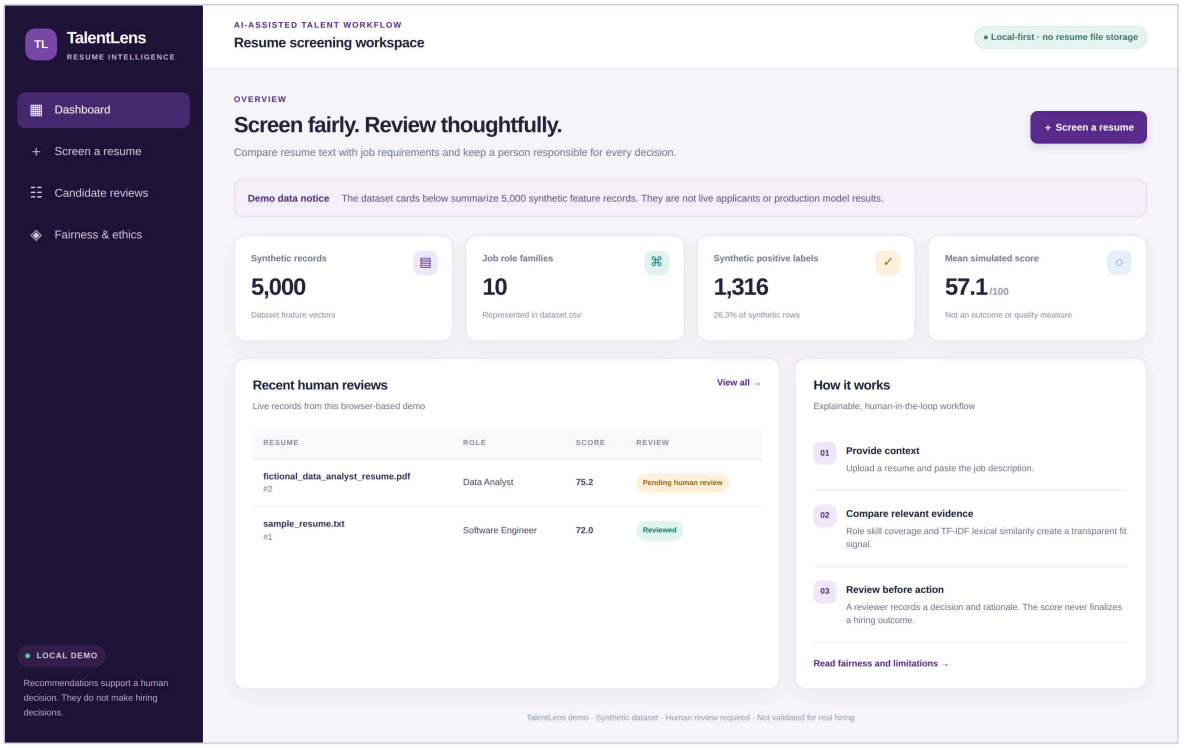


Figure 7. Dashboard of the running application (http://127.0.0.1:5000/): synthetic dataset summary cards and the recent review records created during the live run.

8.1 Live application run (30 October 2026)

CHECK	ACTION	OBSERVED RESULT
Application start	python app.py (Flask development server, 127.0.0.1:5000)	Server started; GET /api/health returned status ok with dataset_rows 5000.
TXT upload	sample_resume.txt, Software Engineer, sample job description	Record #1: fit signal 72.0/100, 7 of 7 skills, TF-IDF similarity 0.1988, "Review for shortlist".
PDF upload	Fictional text-based PDF resume, Data Analyst, short job description	Record #2: text extracted with pdfplumber; fit signal 75.2/100, 6 of 6 skills, TF-IDF similarity 0.2928.
Human review	Record #1: decision "Shortlist" with a written rationale	Saved; status changed from Pending human review to Reviewed (Figure 9).
JSON API	POST /api/score with the sample resume and job description	Returned 72.0 and 0.1988, identical to the web upload result.
CSV export	GET /export.csv	Both records exported with scores, matched skills, status and reviewer decision.
Unit tests	python -m unittest discover -s tests -v	5 tests run, all passed.

Scope: a single-user manual run on the Flask development server with fictional data, captured with headless Chromium. It is not a load, security or usability test, and it does not validate the scorer against real hiring outcomes.

8. Project UI Screenshots - Running Application (continued)

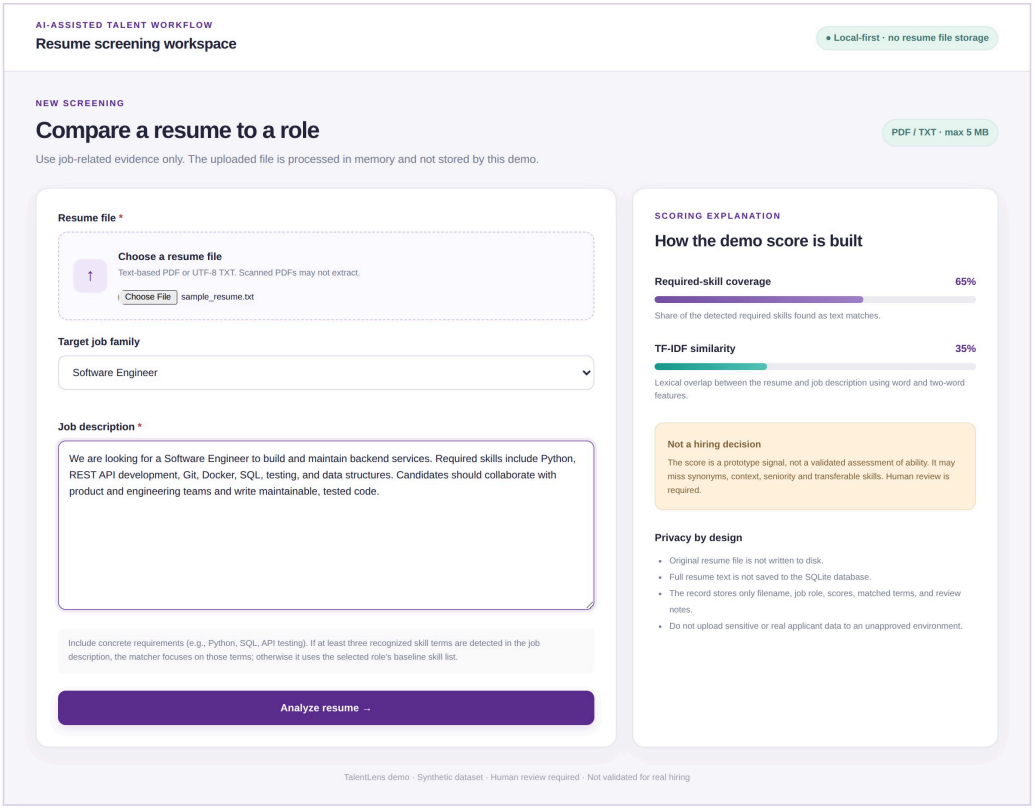


Figure 8. Screening form with sample_resume.txt attached, Software Engineer selected and the sample job description entered, before submission (left navigation bar cropped).

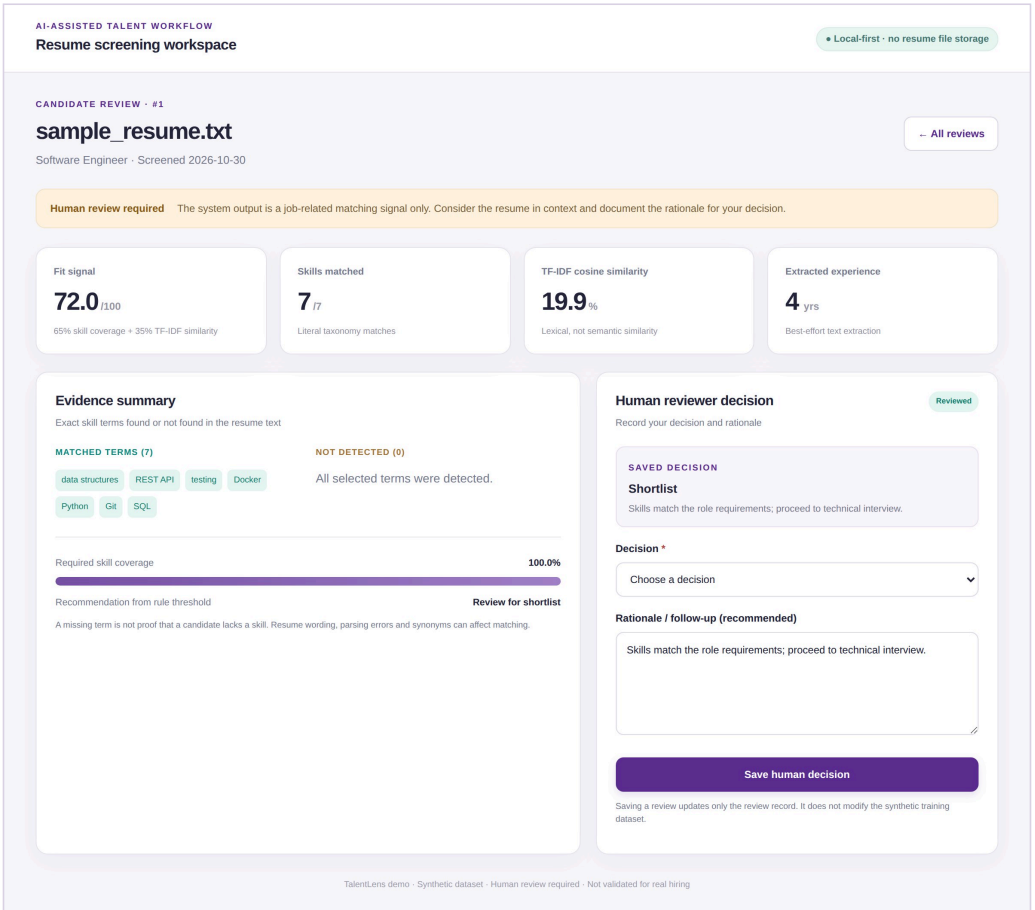


Figure 9. Candidate detail for sample_resume.txt after upload: fit signal 72.0/100, 7 of 7 skills matched, evidence summary and the saved human reviewer decision (left navigation bar cropped).

8. Project UI Screenshots - Running Application (continued)

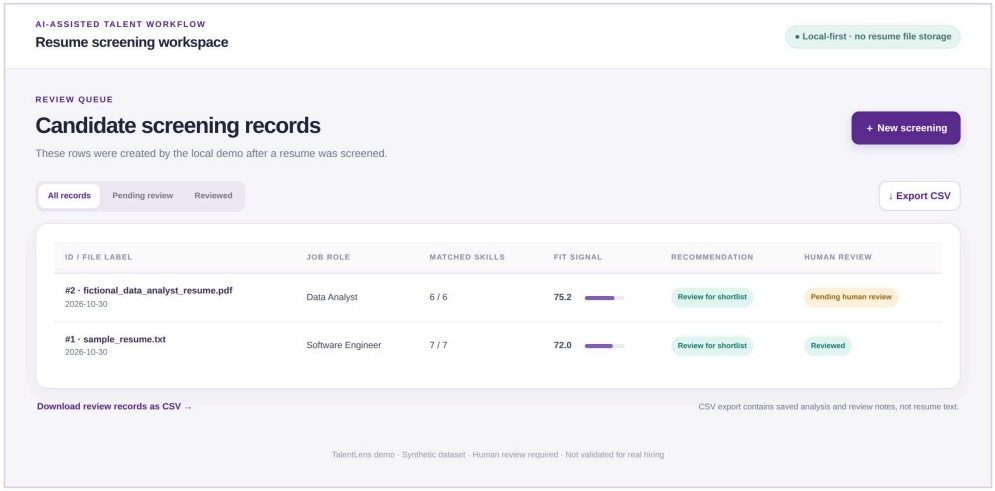


Figure 10. Candidate review queue listing the TXT and PDF test uploads with fit signal, recommendation and review status, plus the CSV export link (left navigation bar cropped).

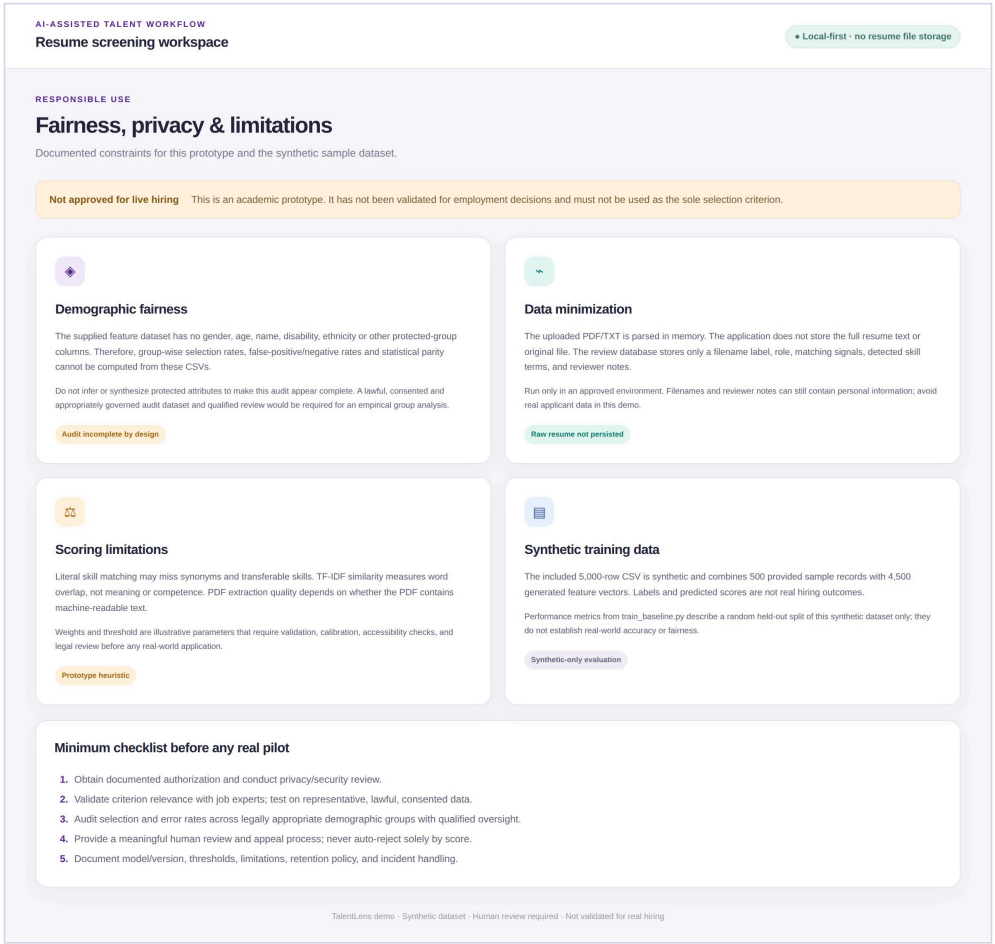


Figure 11. Fairness and ethics page of the application, showing the documented limits of the prototype (left navigation bar cropped).

9. Ethical Review, Fairness and Limitations

9.1 Dataset-level fairness limitation

The feature dataset has no gender, age, name, disability, ethnicity or other protected-group fields. Group-wise selection rates, false-positive/negative rates and statistical parity cannot be computed from the included CSV. The repository appropriately does not infer protected attributes or create synthetic demographic labels to simulate a completed fairness audit.

9.2 Name/pronoun counterfactual perturbation

A narrow input-sensitivity check was run on the live score_resume() function using one fixed software-engineering job description and identical job-related resume content. Ten first-name substitutions were paired (for example, Alex/Jordan, Michael/Michelle, James/Emily); the mean and maximum absolute score difference were both 0.0 points across those tested pairs. A He/She sentence swap produced scores of 79.1 and 79.1, a difference of 0.0 points.

This small counterfactual test suggests that the tested name/pronoun tokens did not alter the score for this one input. It is not a demographic fairness audit, does not cover all names, cultures, document formats or job descriptions, and does not measure group selection or error rates. It must not be interpreted as evidence that the system is unbiased.

9.3 Privacy and responsible use

- The uploaded file and full extracted text are handled in memory and are not persisted by the app.
- SQLite stores selected screening signals, filename label, review notes and reviewer decisions; these fields can still contain personal data.
- Use fictional data for the coursework demo; secure and delete the local database when no longer needed.
- Do not use the score as the sole selection criterion. Reviewers must examine the evidence and document decisions.
- Before any real pilot: complete privacy/security review, validate job-related criteria, evaluate representative data, audit disparate impact, and obtain organizational/legal approval.

10. Conclusion and Future Work

The repository implements an end-to-end academic prototype for extracting text from PDF/TXT resumes, matching detected job requirements, generating an explainable fit signal, storing review metadata, and supporting a human reviewer decision. The package also includes a 5,000-row synthetic feature dataset, a data dictionary, four EDA visualizations, a separate baseline classifier experiment and five passing unit tests.

The most important validation distinction is that the live scorer is a transparent heuristic and the logistic-regression metrics belong to a separate synthetic-data experiment. The 65-point review threshold is illustrative, and the timing numbers cover only the scorer function. The dataset lacks demographic attributes, while a small name/pronoun perturbation test is not a substitute for group-level audit. The current implementation is therefore appropriate as a learning prototype, not as a production or consequential hiring system.

10.1 Future enhancements

- Automate the manual live-run checks in Section 8.1 as browser-level and API integration tests (for example with pytest and Flask's test client).
- Add OCR or an explicit scanned-PDF detection/flagging path; expand parsing tests across multi-column and malformed documents.
- Evaluate the live scorer against a consented, representative, job-validated dataset with reviewer ground truth; calibrate weights and threshold.
- Consider synonym-aware NLP or spaCy matching only after requirements, tests and dependencies are added; update documentation to the implemented feature set.
- Add authentication, role-based access, audit logging, retention/deletion controls and a governed fairness audit before any real-world pilot.

References

[1] HCLTech Career Shaper Programme, “III Year Industry Project - Project Brief,” Project 34, Academic Year 2025-26 (supplied project brief).

[2] Flask Documentation, https://flask.palletsprojects.com/.

[3] pdfplumber Documentation, https://github.com/jsvine/pdfplumber.

[4] scikit-learn, “TfidfVectorizer,” https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[5] scikit-learn, “Model evaluation: quantifying the quality of predictions,” https://scikit-learn.org/stable/modules/model_evaluation.html.

[6] Pandas Documentation, https://pandas.pydata.org/docs/.

[7] NIST, “AI Risk Management Framework,” https://www.nist.gov/itl/ai-risk-management-framework.

Repository: https://github.com/Tan-hue-123/ai-resume-shortlist

Appendix A. AI Tool Disclosure

In line with the Ethical Compliance Declaration, the AI tools used while preparing this project are listed below.

TOOL	USED FOR	HOW THE OUTPUT WAS CHECKED
ChatGPT (OpenAI)	Drafting and revising prototype code, documentation and report/synopsis text; generating the additional synthetic dataset rows.	Dataset checked by eda.py (schema, ranges, missing values, duplicate IDs); code covered by 5 unit tests.
Claude (Anthropic)	Reviewing the package against the project brief and synopsis template; running the Flask app and capturing the real screenshots in report Section 8; re-running eda.py, train_baseline.py and the unit tests; revising report and synopsis wording.	Reported metrics regenerated from the repository scripts; live-run checks recorded in report Section 8.1.

A.1 Statement

No real candidate data is used anywhere in the project: the dataset is synthetic and every test resume is fictional. Reported figures come from running the repository scripts and the live-run checks, not from an AI tool's own claims. AI-generated text and code were reviewed by the learner, who remains responsible for the submitted work.

A.2 Source code repository

GitHub: https://github.com/Tan-hue-123/ai-resume-shortlist

Run locally: pip install -r requirements.txt, then python app.py (http://127.0.0.1:5000). Tests: python -m unittest discover -s tests -v.

Appendix B. Ethical Compliance Declaration

I confirm that:

- ☑ All dataset usage complies with the source licence.
- ☑ No personally identifiable information (PII) is stored without consent.
- ☑ AI tools used for code/writing are disclosed in Appendix A.
- ☑ This project complies with the DPDP Act 2023 where applicable.

Basis: the dataset is the programme-supplied file extended with synthetic rows and holds no resume text or personal fields; test resumes are fictional; the repository carries an MIT licence; AI tools are disclosed in Appendix A.

Learner: Tanishq Tyagi | Roll No.: 2300320100262 | Mentor: Kathiresan | ABES Engineering College, Ghaziabad (4th Year)

Learner Signature: _____ Date: _____